AI Agent for Odoo 20 — Design Description & Roadmap

Oct 9, 2026 · @Hassan

Summary

AI Agent is a new Odoo 20 Community app that lets an administrator set up, govern and watch Hermes Agent without touching config files or a terminal. Odoo stays the control plane: it holds the agent's identity, model, tools, permissions, channels and schedules, and every risky action the agent takes goes through an Odoo approval.

The promise to the admin: **connect the agent in 10 minutes, decide exactly what it may touch, and see everything it did.** It reuses the two apps already designed: Omnichannel (Telegram, WhatsApp, X, LinkedIn) for where the agent talks, and Workflow Automation for when it runs and who approves its actions.

Hermes Agent and how Odoo connects to it

Hermes Agent is Nous Research's open-source, self-improving agent: persistent memory, skills it writes and refines, 40+ built-in tools, MCP support, a messaging gateway, cron, subagents and sandboxed terminal backends (overview). It already exposes everything Odoo needs to manage it, so the app is a control plane, not a fork.

Hermes capability	How the AI Agent app uses it
API server (OpenAI-compatible, bearer key, default port 8642)	Odoo's only link. `/health` for status, `/v1/runs` to start runs, stream events and resolve approvals, `/api/jobs` for schedules, `/api/sessions` for history, `/v1/skills` and `/v1/toolsets` for discovery.
Profiles (separate home, config, memory, skills, cron per agent)	One Odoo **Agent** record = one Hermes profile, e.g. Sales Assistant, HR Helpdesk. Each runs its own gateway and API port.

Hermes capability	How the AI Agent app uses it
MCP client (`mcp_servers`, stdio or HTTP, include/exclude filters)	Odoo publishes an **Odoo MCP server**: one HTTP endpoint per agent, with only the models and operations the admin allowed. Other MCP servers are registered from the same screen.
Security (`approvals.mode` smart/manual/off, timeouts, headless modes, deny rules, user allowlists, Docker isolation)	Shown as plain-language guardrails. Approval requests are routed into Odoo approvals instead of a chat reply.
Cron (natural-language or cron schedules, delivery targets, run history)	Listed and edited in Odoo as Schedules, with results posted to a record's chatter or a channel.
Messaging gateway (Telegram, WhatsApp, Slack, Discord, Email and more)	Optional. By default Odoo Omnichannel owns customer channels and passes conversations to the agent through the API.

Hermes's own rules shape the design: secrets live in `.env` and settings in `config.yaml`, and settings should change through `hermes config set` rather than hand edits. The app therefore never writes files directly; a small connector on the Hermes host applies changes (see Integration outline).

Design principles

Every screen answers one of three admin questions: *what can this agent do, what did it do, and what is waiting for me?*

1. **Safe by default.** A new agent can read nothing until an app is switched on, and every write starts as "needs approval". The admin loosens, never tightens after the fact.
2. **Plain words over config keys.** "Ask me before risky commands" instead of `approvals.mode: manual`. The key is shown in small type for people who need it.
3. **One agent, one card.** Each agent (Hermes profile) has a single page with status, model, tools, channels, schedules and spend. No hunting across menus.
4. **Guided first, detailed later.** A 6-step setup assistant gets a working agent; the Tools, Schedules and Activity screens hold the detail.
5. **Reuse Odoo, don't rebuild it.** Approvals come from Workflow Automation, channels from Omnichannel, users and groups from Odoo security, logs in chatter.

6. **Everything is visible.** Each run shows its prompt, every tool call, each approval and its cost. Nothing the agent does is hidden from the admin.

Information architecture

The app has five menus. Each maps to one admin job, and each has a matching board on the design canvas.

Menu	Admin job	Design board
Agents	See every agent's health, model, access, channels, schedules and budget; act on what needs attention	1 · Agents overview
Configuration	Create or edit an agent with the 6-step assistant: Connect, Identity, Model, Tools & access, Channels, Guardrails & test	2 · Agent setup assistant
Tools & access	Fine-tune per Odoo model (Read, Create, Update, Delete as Off / Allowed / Ask first), connect MCP services, switch Hermes built-in tools, review skills	3 · Tools and access
Automations	Schedules (Hermes cron), the "Ask an agent" workflow step, approvals for agent actions, events and webhooks	4 · Schedules and workflow links
Activity	Every run step by step, approve or deny held actions, see cost, pause all agents	5 · Activity and approvals

Approvers who are not admins never open this app: held actions reach them in **My approvals** (Workflow Automation), email and Telegram.

Key screens

1. Agents overview. A Hermes server strip (address, connected, version, profile count, connector installed, health check) sits on top. Below it, "Needs your attention" tiles: approvals waiting, failed runs, new skills to review. Each agent card lists its Hermes profile, model, Odoo access, channels, schedules and monthly budget, with a status pill (Live, Setup step n of 6, Paused).

2. Agent setup assistant. Six steps with a progress rail, an "About this step" panel and a "Saved in Hermes as" panel that shows the exact keys each step writes, for admins who want to check.

1. Connect: API address, API key, profile (new or existing), Test connection, install the Odoo connector.
2. Identity: name, the Odoo bot user it acts as, languages, tone from the brand kit, and instructions saved to the profile's SOUL.md.
3. Model: provider, model, provider key (to `.env`), fallback model, monthly budget, max steps per run, runs at the same time.
4. Tools & access: one access level per Odoo app (Off, Read, Read and ask before changes, Read and change) plus Hermes built-in tools.
5. Channels: WhatsApp and Telegram from Omnichannel, Discuss, @mention in chatter, website chat, and who may talk to it.
6. Guardrails & test: approval mode as three cards (Ask me / Let a checker decide / Never ask, which is disabled for agents that can write), what unattended runs do, approval timeout, and a test chat that shows tool calls and a held action before Go live.

3. Tools and access. Four tabs. *Odoo records* is a matrix of models × Read/Create/Update/Delete; each cell cycles Off → Allowed → Ask first, plus hidden fields and a record scope. *Connected services* lists MCP servers with status and an add form (URL or local command, auth header, tool filter). *Built-in tools* switches Hermes toolsets and picks the sandbox. *Skills* shows skills the agent wrote, off until approved, with their text to read first. A side panel restates the settings in plain words.

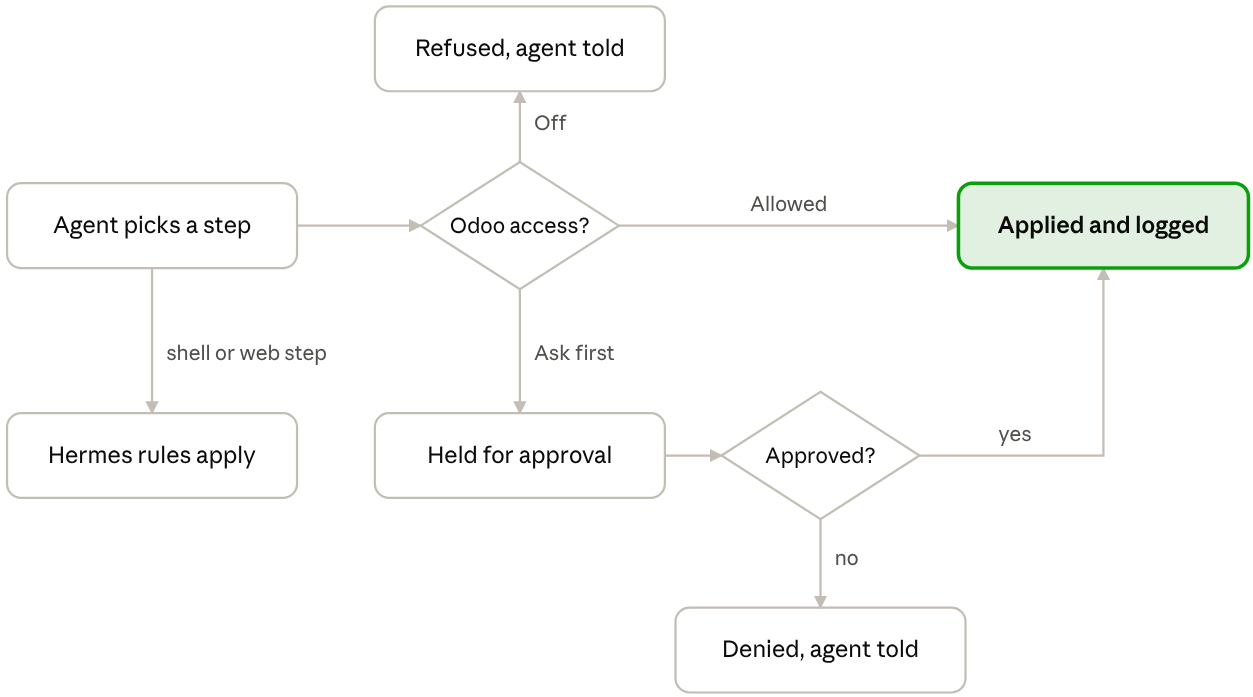
4. Schedules and workflow links. Schedules table (task, agent, when in words and cron, where the result goes, last run) and a new-schedule form that accepts "weekdays at 8am". An "Ask an agent" workflow step whose answer can feed a branch. Events and incoming webhooks that wake an agent.

5. Activity and approvals. Runs on the left, newest and waiting first. On the right: who asked and where, each tool call with its result, and a held action shown as a before/after field table with Approve, Approve and allow from now on, and Deny. Model, tool calls, tokens and cost sit at the bottom. A "Pause all agents" button is always in the header.

Safety, permissions, approvals and audit

Odoo, not the agent, decides whether a change happens. Hermes's own approval setting is a second layer, never the only one.

Every Odoo change the agent asks for passes Odoo's own check



Odoo access check for agent actions · 3 outcomes

The agent asks for a change through the Odoo MCP server. Odoo checks that agent's access matrix: Off refuses, Allowed applies it as the bot user, Ask first holds it and opens an approval that reaches the approver in My approvals, email or Telegram. Shell, file and web steps never touch Odoo, so Hermes's rules govern them, inside a Docker sandbox.

- **Identity.** Each agent is its own Odoo user, so groups and record rules limit it, and every change shows its author in chatter.
- **Least access by default.** New agents start with every app Off and every write on Ask first. "Never ask" mode cannot be picked for an agent that can write to Odoo.
- **Unattended runs.** Schedules and workflows send risky actions to Odoo approval instead of refusing silently. Unanswered approvals are refused after a set time (Hermes `approvals.timeout`, mirrored in Odoo).
- **Data limits.** Hidden fields (cost, margin, internal notes) are removed before data reaches the model. Record scope narrows further, such as one sales team.
- **Secrets.** Provider and API keys are stored encrypted in Odoo and written only to the profile's `.env` through the connector, never to `config.yaml`, following Hermes security guidance.
- **Skills.** Skills the agent writes are off until an admin reads and approves them.

- **Kill switch.** "Pause all agents" stops running steps and blocks new runs.
- **Audit.** Each run keeps the request, every tool call and result, each approval and decision, model, tokens and cost. Logs cannot be edited.

Data model and integration outline

One Odoo module, `ai_agent_hermes`, plus a small connector on the Hermes host. Model names below are proposals.

Model	Holds
`ai.agent.server`	Hermes API address, encrypted API key, version, last health check
`ai.agent`	Name, server, Hermes profile, bot user, persona, provider and model, fallback, budget and limits, approval mode, state
`ai.agent.access`	Per agent and Odoo model: Read / Create / Update / Delete as Off, Allowed or Ask first; hidden fields; record domain
`ai.agent.mcp.server`	Extra MCP services: transport, URL or command, secret headers, tool include/exclude
`ai.agent.skill`	Skill name, origin (admin, bundled, agent), text, state (review, active, rejected)
`ai.agent.schedule`	Prompt, schedule in words and cron, delivery target, Hermes job id
`ai.agent.run` and `ai.agent.run.step`	Source, requester, state, each tool call and result, tokens, cost
`ai.agent.approval`	The held action, before/after values, approver, decision; built on the Workflow Automation approval engine

Odoo to Hermes (Hermes API server, bearer key):

- `/health/detailed` for the status strip; `/v1/skills` and `/v1/toolsets` to list what exists.

- `/v1/runs` to start a run when a customer writes on an Omnichannel channel, someone @mentions the agent or a workflow reaches "Ask an agent"; its SSE events fill the Activity screen; its approval resolution answers Hermes-level approvals from Odoo.
- `/api/jobs` to create, pause and run schedules; `/api/sessions` for transcripts.

Hermes to Odoo (Hermes MCP client):

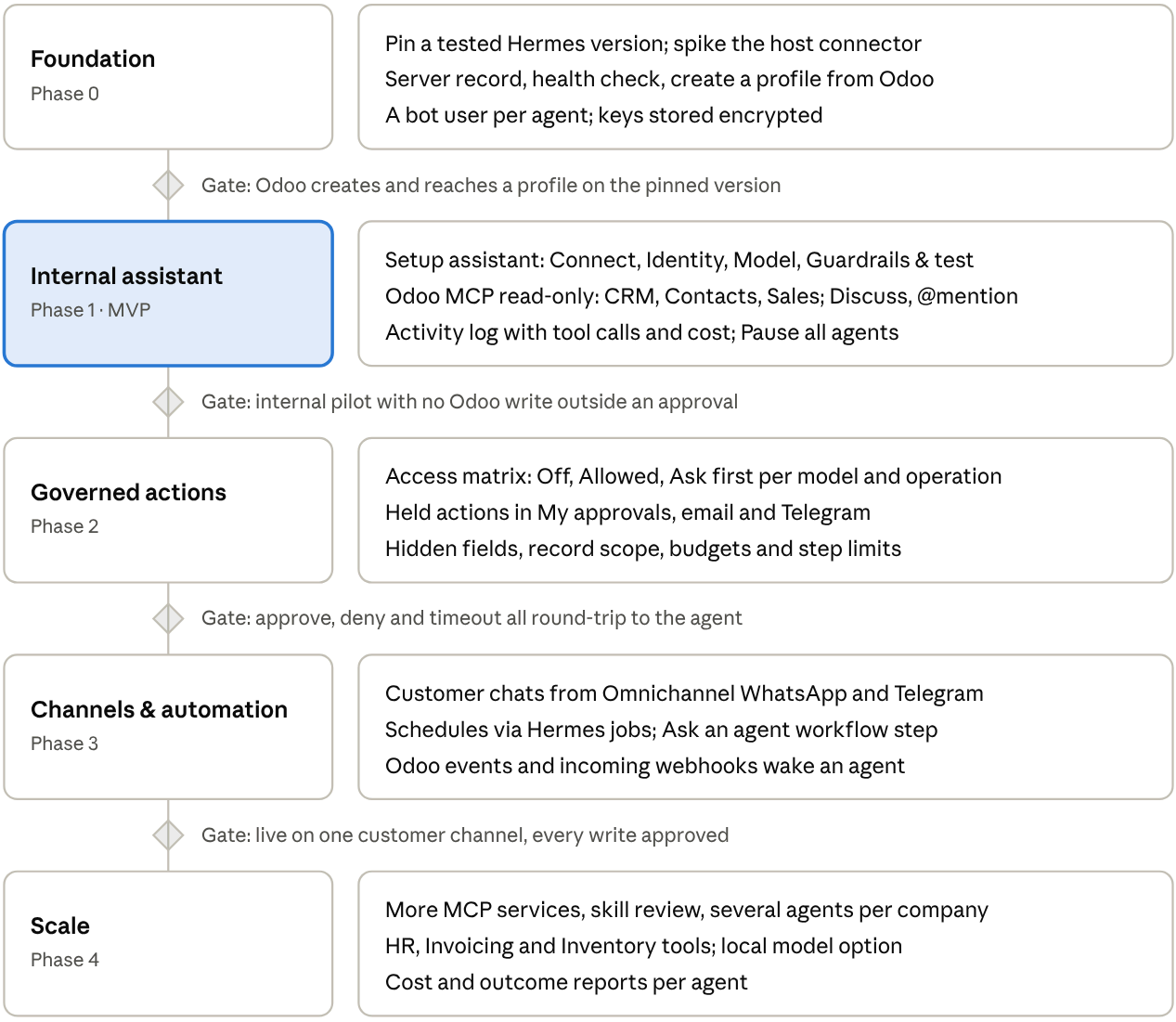
- Odoo serves one HTTP MCP endpoint per agent, such as `/ai_agent/mcp/sales`, with a per-agent token.
- Its tools are generated from the access matrix, such as `odoo_crm_search_leads` and `odoo_sale_create_quotation`. A tool that is Off is never listed.
- An Ask-first call returns "held for approval" with an id. When the approver decides, Odoo posts the result back into the same session so the agent can carry on or tell the person.

Connector on the Hermes host: receives signed settings from Odoo and applies them with Hermes's own commands (`hermes profile create`, `hermes config set`, `.env` for secrets, gateway reload), so files are never hand-edited. Its exact form (small service, Hermes plugin, or dashboard API if one exists) is an open question below.

Roadmap

Ship a safe, read-only internal assistant first (Phase 1), then add writes behind approvals, then customer channels. Each phase ends at a gate that must pass before the next starts.

Five phases, each closed by a gate, from a connected server to many agents



AI Agent roadmap · 5 phases, 4 gates

Phase 1 is the MVP because it proves the connection, the setup assistant and the audit trail without any risk to records. Customer channels wait until Phase 3, after approvals have been proven internally. Phase sizes are left out on purpose: they depend on the Phase 0 connector spike.

Risks, open questions and decisions needed

Risk	Mitigation
Hermes changes quickly; API routes or config keys may move	Pin a tested Hermes version per module release; read `/v1/capabilities` on connect and warn on mismatch
Customer messages try to steer the agent (prompt injection)	Customer-facing agents get no shell or file tools, writes stay Ask first, hidden fields never reach the model
Runaway cost or loops	Monthly budget that pauses the agent, max steps per run, max runs at once
Customer data sent to a model provider (PDPA)	Hidden fields, record scope, and a local or custom-endpoint model option
Approvals raised inside a Hermes gateway chat bypass Odoo	Odoo owns customer channels by default; all Odoo writes are gated by Odoo itself regardless of Hermes mode

Decisions needed:

- [ ] **Channel ownership.** Recommended: Odoo Omnichannel owns WhatsApp and Telegram and passes conversations to Hermes. Alternative: Hermes's own gateway, with a separate bot token per agent.
- [ ] **Connector form.** A small service on the Hermes host, a Hermes plugin, or the Hermes web dashboard if it offers a settings API. Needs a spike against the pinned version.
- [ ] **Hosting.** Hermes on its own VM or container, not the Odoo server.
- [ ] **Default model.** Hosted provider via OpenRouter, or a local model for privacy-sensitive agents such as HR.
- [ ] **Odoo 20 APIs.** Confirm mail, Discuss and approval hooks against the Odoo 20 release before Phase 1 ends.